

Sql Basics

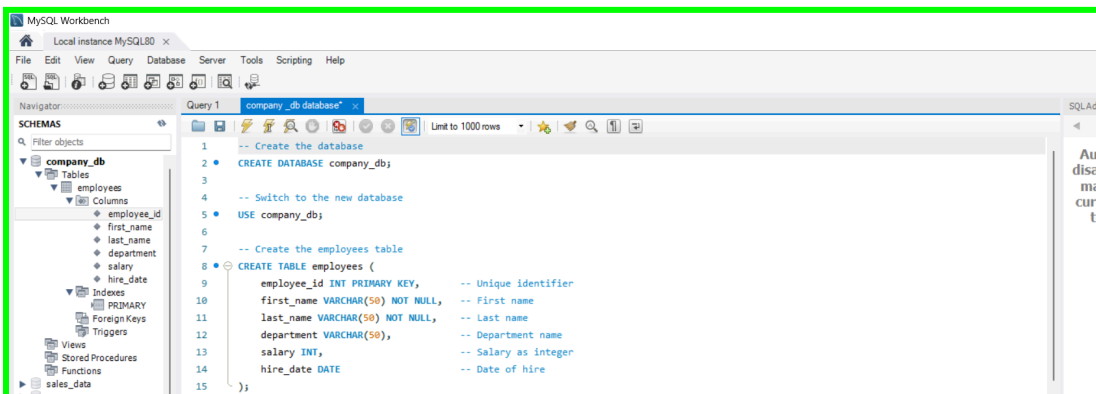
Question no 1:- Create a New Database and Table for Employees Task: Create a new database named company_db and Create a table named employees with the following columns:

Column Name	Data Type	Constraint
employee_id	INT	PRIMARY KEY
first_name	VARCHAR(50)	
last_name	VARCHAR(50)	
department	VARCHAR(50)	
salary	INT	
hire_date	DATE	

Answer:- -- Create the database
CREATE DATABASE company_db;

-- Switch to the new database
USE company_db;

-- Create the employees table
CREATE TABLE employees (
 employee_id INT PRIMARY KEY, -- Unique identifier
 first_name VARCHAR(50) NOT NULL, -- First name
 last_name VARCHAR(50) NOT NULL, -- Last name
 department VARCHAR(50), -- Department name
 salary INT, -- Salary as integer
 hire_date DATE -- Date of hire
);



Question no 2:- Insert Data into Employees Table Task: Insert the following sample records into the employees table,

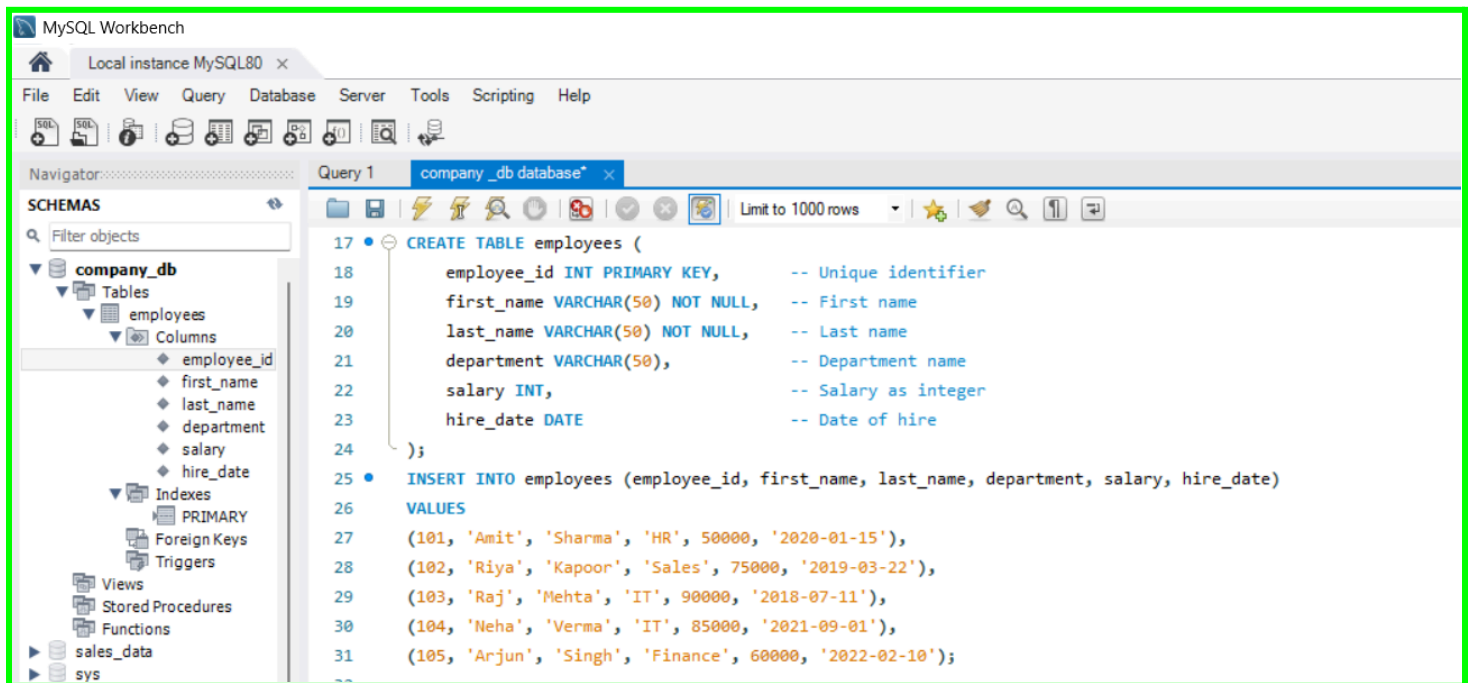
Sql Basics

employee_id	first_name	last_name	department	salary	hire_date
101	Amit	Sharma	Hr	50000	2020-01-15
102	Riya	Kapoor	Sales	75000	2019-03-22
103	Raj	Mehta	It	90000	2018-07-11
104	Neha	Verma	It	85000	2021-09-01
105	Arjun	Singh	Finance	60000	2022-02-10

Answer:- INSERT INTO employees (employee_id, first_name, last_name, department, salary, hire_date)

VALUES

(101, 'Amit', 'Sharma', 'HR', 50000, '2020-01-15'),
(102, 'Riya', 'Kapoor', 'Sales', 75000, '2019-03-22'),
(103, 'Raj', 'Mehta', 'IT', 90000, '2018-07-11'),
(104, 'Neha', 'Verma', 'IT', 85000, '2021-09-01'),
(105, 'Arjun', 'Singh', 'Finance', 60000, '2022-02-10');



Question no3:- Display All Employee Records Sorted by Salary (Lowest to Highest)

Answer:- SELECT *

FROM employees

ORDER BY salary ASC;

Sql Basics

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'company_db' database structure with tables, columns, and indexes. The main query editor shows a query to select all data from the 'employees' table, sorted by salary in ascending order. The result grid shows the output of this query.

```
Query 1: company_db database*
29 (103, 'Raj', 'Mehta', 'IT', 90000, '2018-07-11'),
30 (104, 'Neha', 'Verma', 'IT', 85000, '2021-09-01'),
31 (105, 'Arjun', 'Singh', 'Finance', 60000, '2022-02-10');
32
33 SELECT *
34 FROM employees
35 ORDER BY salary ASC;
```

employee_id	first_name	last_name	department	salary	hire_date
101	Amit	Sharma	HR	50000	2020-01-15
105	Arjun	Singh	Finance	60000	2022-02-10
102	Riya	Kapoor	Sales	75000	2019-03-22
104	Neha	Verma	IT	85000	2021-09-01
103	Raj	Mehta	IT	90000	2018-07-11
NULL	NULL	NULL	NULL	NULL	NULL

Question no:-4 Show Employees Sorted by Department (A–Z) and Salary (High → Low)

Answer:- SELECT *

FROM employees

ORDER BY department ASC, salary DESC;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'company_db' database structure. The main query editor shows a query to select all data from the 'employees' table, sorted by department in ascending order and salary in descending order. The result grid shows the output of this query.

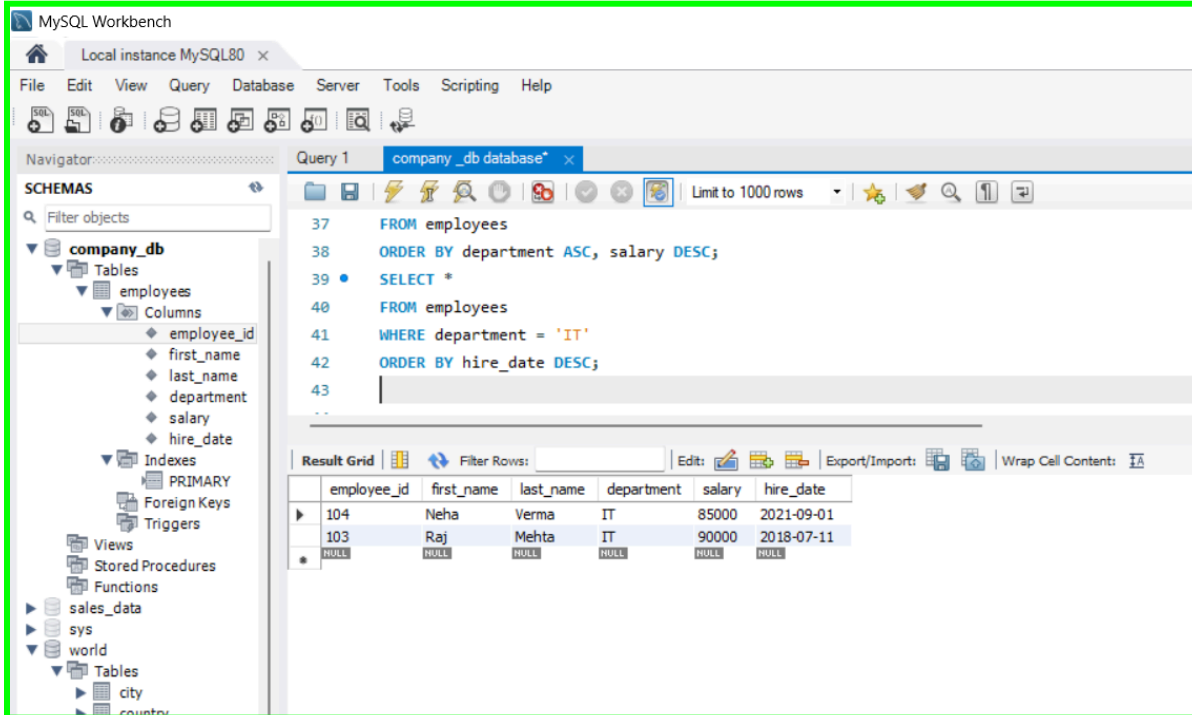
```
Query 1: company_db database*
33 SELECT *
34 FROM employees
35 ORDER BY salary ASC;
36 SELECT *
37 FROM employees
38 ORDER BY department ASC, salary DESC;
39
```

employee_id	first_name	last_name	department	salary	hire_date
105	Arjun	Singh	Finance	60000	2022-02-10
101	Amit	Sharma	HR	50000	2020-01-15
103	Raj	Mehta	IT	90000	2018-07-11
104	Neha	Verma	IT	85000	2021-09-01
102	Riya	Kapoor	Sales	75000	2019-03-22
NULL	NULL	NULL	NULL	NULL	NULL

Sql Basics

Question no5 :- List All Employees in the IT Department, Ordered by Hire Date (Newest First)

Answer:- SELECT *
FROM employees
WHERE department = 'IT'
ORDER BY hire_date DESC;



Question no 6:- Create and Populate a Sales Table Task:

Create a table sales to track sales data:

sale_id	customer_name	amount	sale_date
1	Aditi	1500	2024-08-01
2	Rohan	2200	2024-08-03
3	Aditi	3500	2024-09-05
4	Meena	2700	2024-09-15
5	Rohan	4500	2024-09-25

Answer:- -- Step 1: Create the sales table

```
CREATE TABLE sales (  
  sale_id INT PRIMARY KEY,  
  customer_name VARCHAR(50),  
  amount INT,  
  sale_date DATE
```

Sql Basics

);

-- Step 2: Insert the given records

INSERT INTO sales (sale_id, customer_name, amount, sale_date) VALUES

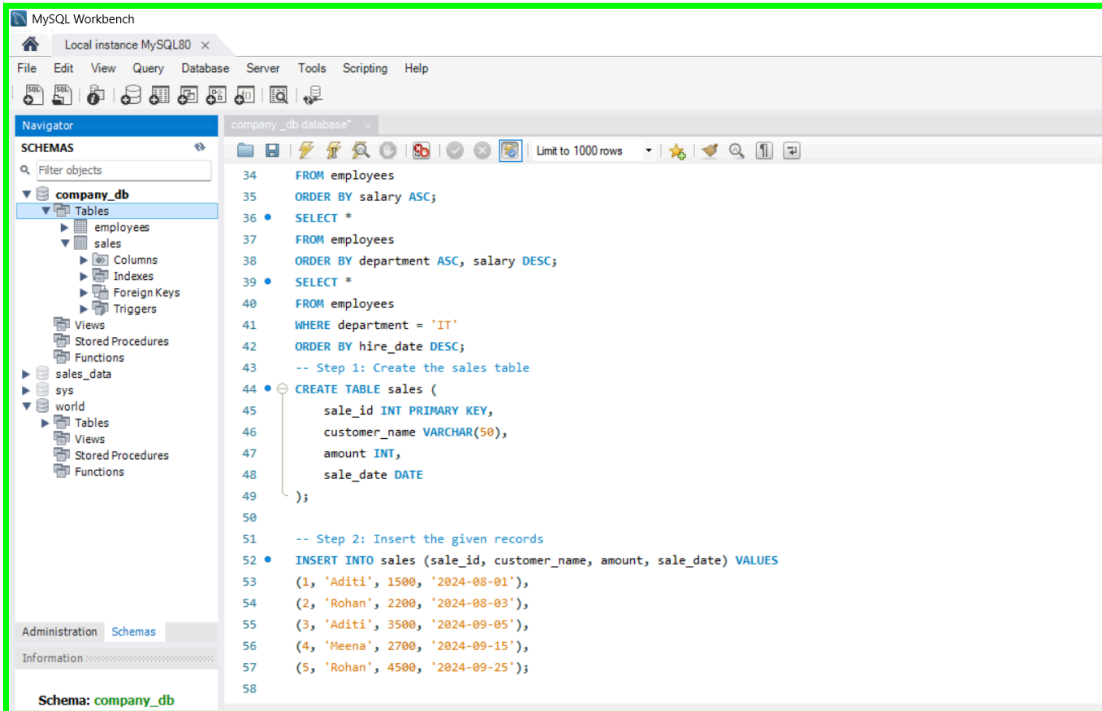
(1, 'Aditi', 1500, '2024-08-01'),

(2, 'Rohan', 2200, '2024-08-03'),

(3, 'Aditi', 3500, '2024-09-05'),

(4, 'Meena', 2700, '2024-09-15'),

(5, 'Rohan', 4500, '2024-09-25');

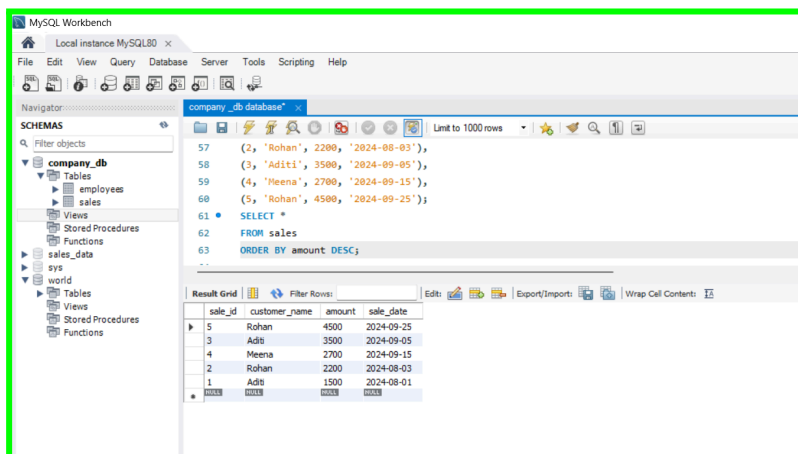


The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'company_db' database schema with tables 'employees' and 'sales'. The main editor window contains the following SQL code:

```
34 FROM employees
35 ORDER BY salary ASC;
36 • SELECT *
37 FROM employees
38 ORDER BY department ASC, salary DESC;
39 • SELECT *
40 FROM employees
41 WHERE department = 'IT'
42 ORDER BY hire_date DESC;
43 -- Step 1: Create the sales table
44 • CREATE TABLE sales (
45     sale_id INT PRIMARY KEY,
46     customer_name VARCHAR(50),
47     amount INT,
48     sale_date DATE
49 );
50
51 -- Step 2: Insert the given records
52 • INSERT INTO sales (sale_id, customer_name, amount, sale_date) VALUES
53 (1, 'Aditi', 1500, '2024-08-01'),
54 (2, 'Rohan', 2200, '2024-08-03'),
55 (3, 'Aditi', 3500, '2024-09-05'),
56 (4, 'Meena', 2700, '2024-09-15'),
57 (5, 'Rohan', 4500, '2024-09-25');
58
```

Question no 7:- Display All Sales Records Sorted by Amount (Highest → Lowest)

Answer:-



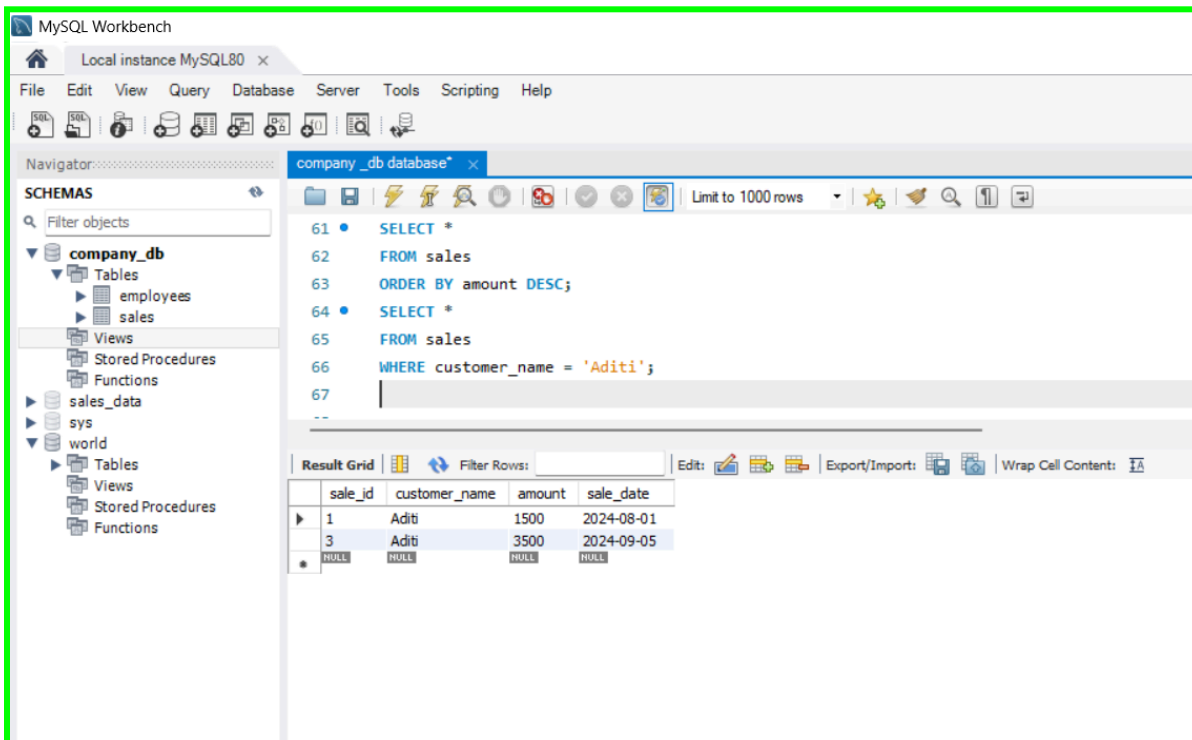
The screenshot shows the MySQL Workbench interface with the same SQL code as before. The 'Result Grid' tab is active, displaying the following data:

sale_id	customer_name	amount	sale_date
5	Rohan	4500	2024-09-25
3	Aditi	3500	2024-09-05
4	Meena	2700	2024-09-15
2	Rohan	2200	2024-08-03
1	Aditi	1500	2024-08-01

Sql Basics

Question no 8:- Show All Sales Made by Customer “Aditi”

Answer:-



Question no 9:- What is the Difference Between a Primary Key and a Foreign Key?

Answer:- A primary key uniquely identifies each record in a table, while a foreign key creates a relationship between two tables by referencing the primary key of another table. In short: *Primary Key = uniqueness inside a table; Foreign Key = linkage across tables.*

Primary Key

- **Definition:** A column (or set of columns) that uniquely identifies each row in a table.
- **Constraints:** Must be **unique** and **not null**.
- **Count per table:** Only **one primary key** allowed.
- **Purpose:** Ensures data integrity by preventing duplicate or null entries.

Example:

sql

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(30)  
);
```

- Here, **emp_id** is the primary key.

Sql Basics

Foreign Key

- **Definition:** A column (or set of columns) in one table that refers to the primary key in another table.
- **Constraints:** Can contain **duplicate values** and may allow **NULLs**.
- **Count per table:** Multiple foreign keys are allowed.
- **Purpose:** Enforces **referential integrity** by linking related data across tables.

Example:

sql

```
CREATE TABLE orders (  
  
    order_id INT PRIMARY KEY,  
  
    emp_id INT,  
  
    order_date DATE,  
  
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)  
  
);
```

- Here, **emp_id** in **orders** is a foreign key referencing **employees**.

Feature	Primary Key	Foreign Key
Purpose	Uniquely identifies each row	Links one table to another
Uniqueness	Must be unique	Can have duplicates
NULL values	Not allowed	Allowed
Count per table	Only one	Multiple allowed
Constraint type	Combination of UNIQUE + NOT NULL	Referential integrity constraint
Example	emp_id in Employees table	emp_id in Orders table referencing Employees

Question no 10:- What Are Constraints in SQL and Why Are They Used?

Sql Basics

Answer:- In SQL, constraints are rules applied to table columns that enforce data integrity, accuracy, and consistency. They prevent invalid data entry, ensure business logic is respected, and maintain reliable relationships between tables.

Constraints Are Used

- **Data Integrity:** Prevents incorrect or inconsistent data (e.g., no duplicate IDs).
- **Accuracy:** Ensures values meet defined conditions (e.g., salary > 0).
- **Reliability:** Maintains valid relationships between tables (e.g., orders linked to valid customers).
- **Automation:** Reduces manual checks by enforcing rules at the database level.

Common Types of SQL Constraints

Constraint	Purpose	Example
NOT NULL	Ensures a column cannot have NULL values.	<code>emp_name VARCHAR(50) NOT NULL</code>
UNIQUE	Ensures all values in a column are distinct.	<code>email VARCHAR(100) UNIQUE</code>
PRIMARY KEY	Combines NOT NULL + UNIQUE; uniquely identifies each row.	<code>emp_id INT PRIMARY KEY</code>
FOREIGN KEY	Links one table to another, enforcing referential integrity.	<code>FOREIGN KEY (dept_id) REFERENCES departments(dept_id)</code>
CHECK	Ensures values meet a condition.	<code>CHECK (salary > 0)</code>
DEFAULT	Provides a default value if none is specified.	<code>status VARCHAR(20) DEFAULT 'Pending'</code>
INDEX	Speeds up data retrieval (not technically a constraint, but often grouped with them).	<code>CREATE INDEX idx_name ON employees(emp_name)</code>

Key Benefits

- **Prevents bad data:** e.g., no negative amounts in a sales table.
- **Supports business rules:** e.g., every sale must have a valid customer.

Sql Basics

- **Improves performance:** queries run faster with indexed constraints.
- **Ensures consistency across tables:** foreign keys prevent orphan records.

Example in Practice

For your `sales` table, constraints could be applied like this:

sql

```
CREATE TABLE sales (  
    sale_id INT PRIMARY KEY,           -- Unique identifier  
    customer_name VARCHAR(50) NOT NULL, -- Must have a name  
    amount INT CHECK (amount > 0),     -- No negative sales  
    sale_date DATE DEFAULT CURRENT_DATE -- Defaults to today if not provided  
);
```

This ensures:

- Each sale has a unique ID.
- Customer names are always provided.
- Sale amounts are positive.
- If no date is entered, today's date is used.

constraints are the “guardrails” of SQL databases—keeping your data clean, consistent, and trustworthy.
